1 Problem Setting and Preliminaries

1.1 Problem Setting

We consider optimization problems in which exact objective function values are unavailable, following the recent literature on error-aware optimization [2, 5, 6, 16, 35, 38]. When both objective values and gradients are heavily corrupted by noise, reliable optimization cannot be guaranteed. Thus, meaningful optimization requires that at least one source of information be sufficiently accurate. This work focuses on problems arising from finite-precision computations or inexact evaluations, where objective function evaluations are subject to non-negligible errors while gradient information can be computed with relatively high accuracy. Settings in which objective values are accurate but gradients are noisy, such as derivative-free optimization or stochastic approximation methods [3, 37], are beyond the scope of this work.

In the following, we formalize the assumptions on function and gradient evaluations. We assume that only an error contaminated function $\overline{f}(x)$ is accessible, satisfying the hybrid absolute-relative error model [20, 21]:

$$\left|f(x)-\overline{f}(x)\right| \leq \varepsilon_{\mathrm{f}} \max(1,|f(x)|), \tag{1}$$

where $0 \leq \varepsilon_{\mathrm{f}} < 1$ is an error rate. This model captures the typical behavior of rounding errors in IEEE 754 floating-point arithmetic [20], and in our numerical experiments, ε_{f} is chosen to range from 10^2 to 10^7 times the machine epsilon. The error is predominantly relative when $|f(x)|$ is large and effectively absolute when $|f(x)|$ is small. Closely related models are also employed in practical stopping criteria [41].

We next specify the assumptions of gradient evaluations. In practice, termination is typically based on a gradient tolerance $\varepsilon_{\mathrm{gtol}}$, and the condition $\left\|\nabla\overline{f}(x)\right\| \leq \varepsilon_{\mathrm{gtol}}$ is used as a stopping criterion. Hence, prior to termination, it holds that $\left\|\nabla\overline{f}(x)\right\| > \varepsilon_{\mathrm{gtol}}$. If the gradient noise level ε_{f} were comparable to or larger than $\varepsilon_{\mathrm{gtol}}$, the stopping criterion could not be satisfied reliably, even when x is sufficiently close to a stationary point. To ensure meaningful termination, the gradient noise must be sufficiently small relative to $\varepsilon_{\mathrm{gtol}}$. Specifically, we assume

$$\left\|\nabla f(x)-\nabla\overline{f}(x)\right\| \ll \varepsilon_{\mathrm{f}} \ll \varepsilon_{\mathrm{gtol}} \leq \left\|\nabla\overline{f}(x)\right\|.$$

Under this regime, the computed gradient $\nabla\overline{f}(x)$ is a sufficiently accurate approximation of the true gradient $\nabla f(x)$ prior to termination. Accordingly, for the purpose of simplifying the theoretical analysis, we adopt the exact-gradient assumption

$$\nabla\overline{f}(x) = \nabla f(x). \tag{2}$$

We emphasize that this assumption is introduced solely as a modeling simplification, rather than as a claim about practical gradient accuracy. Notably, the proposed algorithm does not rely on Wolfe-type conditions, which typically require accurate gradient information at trial points, making it robust to gradient inaccuracies. Our numerical experiments in section 5 include cases where gradients are contaminated by noise, and it also demonstrates that the proposed method remains effective even when eq. (2) is violated.

1.2 Regularized Quasi-Newton Methods

In classical Newton and quasi-Newton methods, line search techniques are commonly used in practice [41]. An alternative approach is to add regularization terms to the Hessian approximation. Recent years have seen renewed interest in regularized Newton-type methods, including cubic regularization schemes [30, 34, 42]. In this work, we focus on quadratic regularization [8, 22, 24, 36, 39, 40, 47].

Let $g_k := \nabla f(x_k)$ and let $B_k \in \mathbb{R}^{n\times n}$ denote an approximate Hessian. In line-search-based quasi-Newton methods, the update is given by

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_k = -B_k^{-1} g_k,$$

where α_k is determined by a line search. In contrast, regularized quasi-Newton methods compute the step as

$$x_{k+1} = x_k + d_k, \quad d_k = -(B_k + \mu_k I)^{-1} g_k,$$

with a regularization parameter $\mu_k \geq 0$. Combining regularization with line search is also possible and has been explored [39].

A fundamental strategy for choosing μ_k in nonconvex optimization was proposed in earlier work on regularized Newton methods [36, 39, 40]. Their formulation sets

$$\mu_k = c_1 \max(0, -\lambda_{\min}(B_k)) + c_2 \|\nabla f(x_k)\|^{\delta}, \tag{3}$$

where $c_1 > 1$, $c_2 > 0$, $\delta \geq 0$ and $\lambda_{\min}(B_k)$ denotes the smallest eigenvalue of B_k. $\|\cdot\|$ is the Euclidean norm. This choice guarantees that $B_k + \mu_k I$ is positive definite, ensuring that d_k is a descent direction. The regularization strategy adopted in this work is closely related in spirit, and we adapt it to noisy evaluation environments.

1.3 Objective-Function-Free Optimization

Objective-Function-Free Optimization (OFFO) refers to a class of methods that avoid explicit evaluations of the objective function and rely solely on gradient information [17, 18]. Such methods are particularly attractive when function values are unreliable due to noise. Similar adaptive trust-region approaches are also discussed [15].

Prominent examples of OFFO include adaptive gradient methods such as AdaGrad [10] and AdaGrad-Norm [43]. Their update rules are

$$x_{k+1}^{(i)} = x_k^{(i)} - \frac{1}{\theta\sqrt{\varsigma + \sum_{j=0}^{k} (g_j^{(i)})^2}} g_k^{(i)}, \qquad (i = 1, 2, \ldots, n)$$

$$x_{k+1} = x_k - \frac{1}{\theta\sqrt{\varsigma + \sum_{j=0}^{k} \|g_j\|^2}} g_k,$$

respectively, where $\varsigma > 0$ and $\theta > 0$ are algorithmic parameters, and $x^{(i)}$ denotes the i-th component of a vector x.

2 Proposed Algorithm

We propose a new algorithm for the problem setting stated in section 1.1. We draw inspiration from the OFFO framework to design robust regularization and scaling strategies. Unlike purely objective-function-free methods, our algorithm exploits function values whenever they are numerically reliable, while smoothly transitioning to gradient-based control when they are not. This hybrid behavior enables improved stability without sacrificing efficiency in practical optimization tasks.

The pseudo-code of our algorithm is given in Algorithm 1, where we note that the minimum of the empty set is defined as $+\infty$ in Line 3. Algorithm 2 is an auxiliary line search algorithm called in Algorithm 1.

Algorithm 1: Noise Tolerant Regularized Quasi-Newton Method

Input: x_0, $0 < k_{\max}$, $0 \le \varepsilon_{\mathrm{f}} < 1$, $0 < c < 1$, $0 < \theta_{\min} \le \theta_{\max}$ and $0 < \varsigma < 1$

1 $k \leftarrow 0$, $g_0 \leftarrow \nabla \overline{f}(x_0)$, $K^0 \leftarrow \emptyset$, $K^+ \leftarrow \emptyset$
2 **for** $k = 0, 1, 2, \ldots, k_{\max} - 1$ **do**
3   **if** $\min_{j \in K^0} (\overline{f}(x_j) - \Delta_j) \ge \overline{f}(x_k)$ **then**
4         $K^0 \leftarrow K^0 \cup \{k\}$
5         $\mu_k \leftarrow 0$
6   **else**
7         $K^+ \leftarrow K^+ \cup \{k\}$
8         $\mu_k \leftarrow \theta_k \sqrt{\varsigma + \sum_{j \in K^+} \|g_j\|^2}$ with $\theta_k \in [\theta_{\min}, \theta_{\max}]$ (section 4.3).
9     $d_k \leftarrow -(B_k + \mu_k I)^{-1} g_k$ with B_k (section 4.1).
10   Find α_k and Δ_k satisfying eq. (4) by Algorithm 2.
11     $x_{k+1} \leftarrow x_k + \alpha_k d_k$, $g_{k+1} \leftarrow \nabla \overline{f}(x_{k+1})$
12 **end**

Algorithm 2: Backtracking Line Search with Relaxed Armijo Condition

Input: $x_k \in \mathbb{R}^n$, $d_k \in \mathbb{R}^n$, $g_k \in \mathbb{R}^n$, $0 < c < 1$ and $0 < \beta_{\min} < \beta_{\max} < 1$

$\alpha_k \leftarrow 1$
$\Delta_k \leftarrow \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_k + \alpha_k d_k))$
while $\overline{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k < \overline{f}(x_k + \alpha_k d_k)$ **do**
    $\alpha_k \leftarrow \beta \alpha_k$ with $\beta \in [\beta_{\min}, \beta_{\max}]$ (section 4.2).
    $\Delta_k \leftarrow \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_k + \alpha_k d_k))$
end
return α_k

In the following, we explain the three main components of Algorithm 1.

Computation of the Regularized Quasi-Newton Direction (Line 9): The first component is the computation of the regularized quasi-Newton direction d_k stated in section 1.2. Given $g_k = \nabla \overline{f}(x_k) = \nabla f(x_k)$, an approximate Hessian B_k, and a regularization parameter μ_k, we compute the search direction $d_k = -(B_k + \mu_k I)^{-1} g_k$. For the theoretical analysis, any choice of B_k satisfying Assumption 3 stated in section 3.1 is admissible, which ensures B_k is positive definite. A practical choice of B_k is discussed in section 4.1.

Relaxed Armijo Condition with an Error-Absorbing Term (Line 10): The second component is the computation of the step size α_k using a relaxed Armijo condition. We compute α_k satisfying the relaxed Armijo condition

$$\begin{cases} \overline{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k \geq \overline{f}(x_k + \alpha_k d_k), \\ \Delta_k = \dfrac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_k + \alpha_k d_k)) \end{cases} \tag{4}$$

where Δ_k is the error-absorbing term that is recomputed for each α_k. This relaxation guarantees the existence of $\alpha_k \in (0,1]$ even in the presence of numerical errors in $\overline{f}$. When d_k is a descent direction ($g_k^\top d_k \leq 0$), eq. (4) ensures that a temporary increase of $\overline{f}$ is at most Δ_k. Since Δ_k depends on $-\overline{f}(x_k + \alpha_k d_k)$ but not on $\overline{f}(x_k + \alpha_k d_k)$, the value of Δ_k is independent of how much $\overline{f}(x_k + \alpha_k d_k)$ increases, preventing unbounded growth of Δ_k.

Adaptive Update of the Regularization Parameter (Lines 3–8): The third component is the adaptive update of the regularization parameter μ_k. A larger μ_k yields a more conservative step, while a smaller μ_k allows a more aggressive quasi-Newton step, especially when $\mu_k = 0$. In practical settings, setting $\mu_k = 0$ is preferred, since this allows us to fully exploit the quasi-Newton approximation and typically leads to practically faster convergence. Let us partition the iteration indices $K := \{0, 1, 2, \ldots, k_{\max} - 1\}$ into two sets:

$$K^0 := \{k \in K \mid \mu_k = 0\}, \quad K^+ := \{k \in K \mid \mu_k > 0\}. \tag{5}$$

We set $\mu_k = 0$ if and only if the following condition is satisfied:

$$\min_{j \in K^0, j<k} \left(\overline{f}(x_j) - \Delta_j\right) \geq \overline{f}(x_k). \tag{6}$$

Note that $k = 0$ satisfies eq. (6) since K^0 is empty. This condition restricts $\mu_k = 0$ to iterations for which a sufficient decrease in $\overline{f}$ has been observed. Thereby, while $\overline{f}$ can temporarily increase, this condition theoretically ensures that $\overline{f}$ does not grow unboundedly or oscillate indefinitely.

Otherwise, to ensure convergence to first-order stationary points, we apply an OFFO-based update

$$\mu_k = \theta_k \sqrt{\varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2}, \tag{7}$$

as described in section 1.3. Since OFFO schemes do not rely on function values, this update is robust to numerical errors in $\overline{f}$.

3 Theoretical Analysis

In this section, we establish the global convergence properties of Algorithm 1 under the problem setting in section 1.1, namely, eqs. (1) and (2).

3.1 Assumptions

Throughout this section, we make the following three assumptions. The first assumption guarantees that the objective function is bounded below.

Assumption 1 *The function f is bounded below; that is, for all $x \in \mathbb{R}^n$,*

$$f(x) \geq f^* := \inf_{x \in \mathbb{R}^n} f(x) > -\infty.$$

By the triangle inequality and eq. (1), we have $\overline{f}(x) \geq f(x) - \varepsilon_{\mathrm{f}} \max(1, |f(x)|)$. Since the function $g(t) = t - \varepsilon_{\mathrm{f}} \max(1, |t|)$ is non-decreasing for $0 \leq \varepsilon_{\mathrm{f}} < 1$, it follows that $\inf_x g(f(x)) = g(f^*)$. Consequently, we can also lower bound $\overline{f}$ as

$$\overline{f}(x) \geq \overline{f}^* := \inf_{x \in \mathbb{R}^n} (f(x) - \varepsilon_{\mathrm{f}} \max(1, |f(x)|)) = f^* - \varepsilon_{\mathrm{f}} \max(1, |f^*|) > -\infty. \tag{8}$$

The second assumption ensures smoothness of the objective function.

Assumption 2 *The function $f \colon \mathbb{R}^n \to \mathbb{R}$ is L-smooth, meaning that there exists a constant $L > 0$ such that for all $x, y \in \mathbb{R}^n$,*

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|y - x\|^2.$$

The third assumption requires the matrices $\{B_k\}_{k=0}^{k_{\max}-1}$ to be uniformly bounded and uniformly positive definite.

Assumption 3 *There exist constants $0 < m \leq M$ such that all B_k satisfy*

$$mI \preceq B_k \preceq MI.$$

As we will discuss in section 4, the standard BFGS update with a slight modification ensures that Assumption 3 holds in practice. By Assumption 3, we can derive two useful bounds involving g_k and d_k. Using $d_k = -(B_k + \mu_k I)^{-1} g_k$ (Line 9) and Assumption 3 asserting $(m + \mu_k)I \preceq B_k + \mu_k I \preceq (M + \mu_k)I$, we obtain the following two bounds:

$$-g_k^\top d_k = g_k^\top (B_k + \mu_k I)^{-1} g_k \geq \frac{1}{M + \mu_k} \|g_k\|^2, \tag{9}$$

$$\|d_k\|^2 = \left\|(B_k + \mu_k I)^{-1} g_k\right\|^2 \leq \frac{1}{(m + \mu_k)^2} \|g_k\|^2. \tag{10}$$

3.2 Error Bound on Function Evaluations

In the following, we define the actual and observed function decreases as

$$\delta_k := f(x_k) - f(x_{k+1}), \tag{11}$$
$$\overline{\delta}_k := \overline{f}(x_k) - \overline{f}(x_{k+1}). \tag{12}$$

We first prove a useful property of the error-absorbing term Δ_k defined in eq. (4). The following lemma asserts that the error between the actual values difference δ_k and the observed values difference $\overline{\delta}_k$ can be bounded by Δ_k.

Lemma 1 *If x_k and x_{k+1} satisfies $f(x_{k+1}) \leq f(x_k)$, it holds that*

$$\delta_k \leq \overline{\delta}_k + \Delta_k. \tag{13}$$

Proof We start from a general observation. We claim that, for all $x \in \mathbb{R}^n$,

$$\begin{cases} \max(1, f(x)) \leq \frac{1}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x)), \\ \max(1, -f(x)) \leq \frac{1}{1-\varepsilon_{\mathrm{f}}} \max(1, -\overline{f}(x)). \end{cases} \tag{14}$$

We only consider the case with the plus sign, as the negative case can be treated analogously. If $f(x) \leq 1$, then $\max(1, f(x)) = 1 \leq \frac{1}{1-\varepsilon_{\mathrm{f}}}$ and the inequality is trivial. If $f(x) > 1$, eq. (1) implies

$$\left|f(x) - \overline{f}(x)\right| \leq \varepsilon_{\mathrm{f}} \max(1, |f(x)|) = \varepsilon_{\mathrm{f}} f(x). \tag{15}$$

Since $\varepsilon_{\mathrm{f}} < 1$, eq. (15) implies that $f(x)$ and $\overline{f}(x)$ have the same sign. Moreover, by the triangle inequality, $|f(x)| - \left|\overline{f}(x)\right| \leq \varepsilon_{\mathrm{f}} |f(x)|$, which yields $\max(1, f(x)) = f(x) \leq \frac{1}{1-\varepsilon_{\mathrm{f}}} \overline{f}(x)$. Combining the two cases completes the proof of eq. (14).

With these preparations, we now prove the lemma. For all $k \in K$, we have

$$\begin{aligned} \left|\delta_k - \overline{\delta}_k\right| &= \left|(f(x_k) - \overline{f}(x_k)) - (f(x_{k+1}) - \overline{f}(x_{k+1}))\right| && \text{(rearrangement)} \\ &\leq \varepsilon_{\mathrm{f}} \max(1, |f(x_k)|) + \varepsilon_{\mathrm{f}} \max(1, |f(x_{k+1})|) && \text{(definition in eq. (1))} \\ &\leq 2\varepsilon_{\mathrm{f}} \max(1, |f(x_k)|, |f(x_{k+1})|) && \text{(max property)} \end{aligned} \tag{16}$$

From the condition of this lemma ($f(x_{k+1}) \leq f(x_k)$), this can be further bounded as

$$\begin{aligned} 2\varepsilon_{\mathrm{f}} \max(1, |f(x_k)|, |f(x_{k+1})|) &= 2\varepsilon_{\mathrm{f}} \max(1, f(x_k), -f(x_{k+1})) && (f(x_{k+1}) \leq f(x_k)) \\ &\leq \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_{k+1})) && \text{(eq. (14))} \\ &= \Delta_k, && \text{(eq. (4))} \end{aligned}$$

which yields the desired inequality by the triangle inequality. □

We also derive the following lemma for later use.

Lemma 2 *If x_k and x_{k+1} satisfy the relaxed Armijo condition (eq. (4)) and d_k is a descent direction (i.e., $g_k^\top d_k \le 0$), it holds that*

$$\overline{\delta}_k \le \delta_k + \frac{1+\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\Delta_k.$$

Proof By condition of this lemma and eq. (4), we have

$$\overline{f}(x_k) - \overline{f}(x_{k+1}) \ge -c\alpha_k g_k^\top d_k - \Delta_k \ge -\Delta_k. \tag{17}$$

Using eqs. (14) and (17), the right-hand side of eq. (16) can be further bounded as

$$\begin{aligned}
2\varepsilon_{\mathrm{f}} \max(1, |f(x_k)|, |f(x_{k+1})|) &\le \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_k), \overline{f}(x_{k+1}), -\overline{f}(x_{k+1})) \\
&\le \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k) + \Delta_k, -\overline{f}(x_{k+1}) + \Delta_k) \\
&\le \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \big(\max(1, \overline{f}(x_k), -\overline{f}(x_{k+1})) + \Delta_k\big).
\end{aligned}$$

Combining this with eq. (16) and the definition of Δ_k in eq. (4) yields

$$\left|\delta_k - \overline{\delta}_k\right| \le \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \big(\max(1, \overline{f}(x_k), -\overline{f}(x_{k+1})) + \Delta_k\big) = \left(1 + \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\right)\Delta_k = \frac{1+\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\Delta_k.$$

Applying the triangle inequality completes the proof. □

3.3 Backtracking Stage

We next analyze the line search in Algorithm 2. Recall that we assume that the parameters in Algorithm 2 satisfy $0 < c < 1$ and $0 < \beta_{\min} < \beta_{\max} < 1$.

Proposition 1 *Under Assumptions 2 and 3, the backtracking procedure in Algorithm 2 terminates whenever the step size α_k satisfies*

$$\alpha_k \le \frac{2(1-c)m}{L}. \tag{18}$$

Proof Under the L-smoothness of f (Assumption 2), we always have

$$\begin{aligned}
f(x_k) - f(x_k + \alpha_k d_k) &\ge -\alpha_k g_k^\top d_k - \frac{L}{2}\alpha_k^2 \|d_k\|^2 \\
&= -c\alpha_k g_k^\top d_k + \alpha_k \left(-(1-c) g_k^\top d_k - \frac{L\alpha_k}{2}\|d_k\|^2\right).
\end{aligned} \tag{19}$$

From Line 9 of Algorithm 1, it follows that $g_k = -(B_k + \mu_k I)d_k$. Then, from Assumption 3 and $\mu_k \ge 0$, we have

$$-g_k^\top d_k = d_k^\top (B_k + \mu_k I) d_k \ge (m + \mu_k)\|d_k\|^2 \ge 0. \tag{20}$$

Thus, when α_k satisfies the bound in eq. (18), we have

$$-(1-c)g_k^\top d_k - \frac{L\alpha_k}{2}\|d_k\|^2 \geq \left((1-c)m - \frac{L\alpha_k}{2}\right)\|d_k\|^2 \geq 0. \tag{21}$$

Combining eqs. (19) and (21) yields

$$f(x_k) - f(x_k + \alpha_k d_k) \geq -c\alpha_k g_k^\top d_k \geq 0.$$

Hence, the condition of Lemma 1 is satisfied with $x_{k+1} = x_k + \alpha_k d_k$ and thus we obtain $\delta_k \leq \overline{\delta}_k + \Delta_k$, i.e.,

$$-c\alpha_k g_k^\top d_k \leq f(x_k) - f(x_k + \alpha_k d_k) \leq \overline{f}(x_k) - \overline{f}(x_k + \alpha_k d_k) + \Delta_k.$$

This inequality means that the relaxed Armijo condition (eq. (4)) is satisfied and thus the backtracking procedure terminates. This completes the proof. $\square$

Now, let us analyze the behavior of Algorithm 2 using Proposition 1. Define

$$\overline{\alpha} := \beta_{\min} \min\left(\frac{2(1-c)m}{L}, 1\right) > 0. \tag{22}$$

The following corollary summarizes two immediate consequences: the backtracking procedure terminates after finitely many rejections, and the resulting step size is uniformly bounded by $\overline{\alpha}$.

Corollary 1 *Under Assumptions 2 and 3, the backtracking line search in Algorithm 2 terminates after finitely many rejections, and the step size α_k satisfies*

$$\alpha_k \geq \overline{\alpha}.$$

Proof Let r denote the number of backtracking rejections. Initially, $\alpha_k = 1$, and each rejection multiplies α_k by $\beta \leq \beta_{\max} < 1$. Hence, after r rejections, $\alpha_k \leq \beta_{\max}^r$ holds. By Proposition 1, if

$$\alpha_k \leq \beta_{\max}^r \leq \frac{2(1-c)m}{L}, \tag{23}$$

the backtracking procedure must terminate. Since $0 < c < 1$ and $0 < m$, the right-hand side is positive, and thus r is finite. We next establish the lower bound of α_k. For the case $r = 0$, $\alpha_k = 1 \geq \overline{\alpha}$ by $\beta_{\min} < 1$. Otherwise, let α_k' denote the last rejected step size. By Proposition 1, we have

$$\alpha_k' > \frac{2(1-c)m}{L} \geq \min\left(\frac{2(1-c)m}{L}, 1\right). \tag{24}$$

The accepted step size α_k is chosen from the interval $[\beta_{\min}\alpha_k', \beta_{\max}\alpha_k']$, and thus

$$\alpha_k \geq \beta_{\min}\alpha_k' > \beta_{\min} \min\left(\frac{2(1-c)m}{L}, 1\right) = \overline{\alpha}. \tag{25}$$

This completes the proof. $\square$

3.4 Bound for the case $\mu_k = 0$

In this subsection, we bound the sum of $\|g_k\|^2$ over $k \in K^0$. Recall that K^0 be a set of iteration indices k such that $\mu_k = 0$ in Algorithm 1 as defined in eq. (5).

To this end, we first establish that the sum of Δ_k over $k \in K^0$ is bounded, where Δ_k is the error-absorbing term in the relaxed Armijo condition (eq. (4)). To this purpose, we introduce the following constant:

$$\Delta_{\max} := \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_0), -\overline{f}^*). \tag{26}$$

Lemma 3 *Under Assumptions 1 to 3, the sum of Δ_k over $k \in K^0$ satisfies*

$$\sum_{k \in K^0} \Delta_k \le \overline{f}(x_0) - \overline{f}^* + \Delta_{\max}.$$

Proof Let us define the last index of $\left|K^0\right|$ as ℓ, and enumerate the elements of K^0 in increasing order as $K^0 = (k_i)_{i=0}^{\ell}$. For all $i < \ell$, using eq. (6) with $j = k_i$ and $k = k_{i+1}$, we obtain

$$\Delta_{k_i} \le \overline{f}(x_{k_i}) - \overline{f}(x_{k_{i+1}}). \tag{27}$$

Summing up eq. (27) over $i < \ell$ and adding the case k_ℓ yields

$$\sum_{k \in K^0} \Delta_k \le \sum_{i=0}^{\ell-1} (\overline{f}(x_{k_i}) - \overline{f}(x_{k_{i+1}})) + \Delta_{k_\ell} = \overline{f}(x_0) - \overline{f}(x_{k_\ell}) + \Delta_{k_\ell}. \tag{28}$$

It also holds that $\overline{f}(x_0) \ge \overline{f}(x_{k_\ell})$ from eq. (6) with $j = 0$ and $k = k_\ell$. We also have $\overline{f}(x_{k_\ell}) \ge \overline{f}^*$ from Assumption 1 and eq. (8). Thus, eq. (28) can be further bounded as

$$\begin{aligned}
\sum_{k \in K^0} \Delta_k &\le \overline{f}(x_0) - \overline{f}(x_{k_\ell}) + \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_{k_\ell}), -\overline{f}(x_{k_\ell+1})) && \text{(eq. (4))} \\
&\le \overline{f}(x_0) - \overline{f}^* + \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_0), -\overline{f}^*) && \text{(eq. (8))} \\
&= \overline{f}(x_0) - \overline{f}^* + \Delta_{\max}. && \text{(eq. (26))}
\end{aligned}$$

This completes the proof. □

Using this lemma, we can bound the sum of $\|g_k\|^2$ over $k \in K^0$ using $\delta_k = f(x_k) - f(x_{k+1})$ in eq. (11) as follows.

Lemma 4 *Under Assumptions 1 to 3, the sum of δ_k over $k \in K^0$ satisfies*

$$\sum_{k \in K^0} \delta_k \ge \frac{c\overline{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 - \frac{2}{1-\varepsilon_{\mathrm{f}}} \left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max} \right).$$

Proof The relaxed Armijo condition (eq. (4)) at iteration $k \in K^0$ yields

$$\begin{aligned}
\overline{\delta}_k + \Delta_k &= \overline{f}(x_k) - \overline{f}(x_{k+1}) + \Delta_k && \text{(eq. (12))} \\
&\geq -c\alpha_k g_k^\top d_k && \text{(eq. (4))} \\
&\geq \frac{c\overline{\alpha}}{M+\mu_k}\|g_k\|^2 && \text{(Corollary 1 and eq. (9))} \\
&= \frac{c\overline{\alpha}}{M}\|g_k\|^2. && (\mu_k = 0 \text{ since } k \in K^0) \qquad (29)
\end{aligned}$$

Since the conditions of Lemma 2 are always satisfied, we also have

$$\overline{\delta}_k + \Delta_k \leq \delta_k + \frac{1+\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\Delta_k + \Delta_k = \delta_k + \frac{2}{1-\varepsilon_{\mathrm{f}}}\Delta_k. \qquad (30)$$

Then, combining eqs. (29) and (30) and summing up over $k \in K^0$ gives

$$\sum_{k\in K^0} \frac{c\overline{\alpha}}{M}\|g_k\|^2 \leq \sum_{k\in K^0} \delta_k + \frac{2}{1-\varepsilon_{\mathrm{f}}}\sum_{k\in K^0}\Delta_k \leq \sum_{k\in K^0}\delta_k + \frac{2}{1-\varepsilon_{\mathrm{f}}}\left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max}\right),$$

where the last inequality follows from Lemma 3. This completes the proof. □

3.5 Bound for the case $\mu_k > 0$

Next, we show that the sum of $\|g_k\|^2$ over $k \in K^+$ is bounded. Recall that K^+ be a set of iteration indices k such that $\mu_k > 0$ as defined in eq. (5), and that μ_k is defined as $\theta_k\sqrt{\varsigma + \sum_{j\in K^+,\ j<k}\|g_j\|^2}$ with $\theta_k \in [\theta_{\min}, \theta_{\max}]$ in Line 8 of Algorithm 1.

Before proceeding to the main proof, we present standard inequalities in the AdaGrad-Norm algorithm analysis [43, Lemma 3.2], [18, Lemma 3.1]. Its proof is deferred to **??**.

Lemma 5 *In Algorithm 1, we have*

$$\sum_{k\in K^+}\frac{\|g_k\|^2}{\mu_k^2} \leq \frac{1}{\theta_{\min}^2}\left(\ln\left(\varsigma + \sum_{k\in K^+}\|g_k\|^2\right) - \ln(\varsigma)\right),$$
$$\sum_{k\in K^+}\frac{\|g_k\|^2}{\mu_k} \geq \frac{1}{\theta_{\max}}\left(\sqrt{\varsigma + \sum_{k\in K^+}\|g_k\|^2} - \sqrt{\varsigma}\right).$$

We bound the sum of $\|g_k\|^2$ over $k \in K^+$ using δ_k defined in eq. (11). Now, we define the following constants for later use:

$$C_1 := \frac{\overline{\alpha}}{\theta_{\max}} \qquad C_2 := \frac{M\overline{\alpha} + L/2}{\theta_{\min}^2} \qquad (31)$$

Lemma 6 *Under Assumptions 2 and 3, the sum of δ_k over $k \in K^+$ satisfies*

$$\sum_{k\in K^+}\delta_k \geq C_1\left(\sqrt{\varsigma + \sum_{k\in K^+}\|g_k\|^2} - \sqrt{\varsigma}\right) - C_2\left(\ln\left(\varsigma + \sum_{k\in K^+}\|g_k\|^2\right) - \ln(\varsigma)\right)$$

Proof From the L-smoothness (Assumption 2), for $k \in K^+$ we have

$$\begin{aligned}
\delta_k &= f(x_k) - f(x_k + \alpha_k d_k) && (\text{eq. (11) and } x_{k+1} = x_k + \alpha_k d_k)\\
&\geq -\alpha_k g_k^\top d_k - \frac{\alpha_k^2 L}{2}\|d_k\|^2 && (\text{Assumption 2})\\
&\geq \left(\frac{\alpha_k}{M+\mu_k} - \frac{\alpha_k^2 L}{2(m+\mu_k)^2}\right)\|g_k\|^2 && (\text{eqs. (9) and (10)})\\
&\geq \left(\frac{\overline{\alpha}}{M+\mu_k} - \frac{L}{2(m+\mu_k)^2}\right)\|g_k\|^2. && (\text{Corollary 1 and } \alpha_k \leq 1) \qquad (32)
\end{aligned}$$

We can further bound the terms in eq. (32) as

$$\frac{\overline{\alpha}}{M+\mu_k} = \frac{\overline{\alpha}}{\mu_k}\frac{1}{1+\frac{M}{\mu_k}} \geq \frac{\overline{\alpha}}{\mu_k}\left(1-\frac{M}{\mu_k}\right) = \frac{\overline{\alpha}}{\mu_k} - \frac{M\overline{\alpha}}{\mu_k^2}, \qquad \frac{L}{2(m+\mu_k)^2} \leq \frac{L}{2\mu_k^2}. \tag{33}$$

Thus, applying eq. (33) to eq. (32) yields

$$\delta_k \geq \left(\frac{\overline{\alpha}}{\mu_k} - \frac{M\overline{\alpha}+L/2}{\mu_k^2}\right)\|g_k\|^2. \tag{34}$$

Summing up eq. (34) over $k \in K^+$ and applying Lemma 5 yield

$$\begin{aligned}
&\sum_{k\in K^+}\delta_k\\
&\geq \sum_{k\in K^+}\left(\frac{\overline{\alpha}}{\mu_k} - \frac{M\overline{\alpha}+L/2}{\mu_k^2}\right)\|g_k\|^2\\
&\geq \frac{\overline{\alpha}}{\theta_{\max}}\left(\sqrt{\varsigma + \sum_{k\in K^+}\|g_k\|^2} - \sqrt{\varsigma}\right) - \frac{M\overline{\alpha}+L/2}{\theta_{\min}^2}\left(\ln\left(\varsigma + \sum_{k\in K^+}\|g_k\|^2\right) - \ln(\varsigma)\right),
\end{aligned}$$

which yields the desired result. □

3.6 Global Convergence Result

Finally, we combine the results for $k \in K^0$ (Lemma 4) and $k \in K^+$ (Lemma 6) to derive a bound on the total sum of $\|g_k\|^2$. We define the constant F as follows:

$$F := f(x_0) - f^* + \frac{2}{1-\varepsilon_{\mathrm{f}}}\left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max}\right) + C_1\sqrt{\varsigma} - C_2\ln(\varsigma). \tag{35}$$

Then, we can obtain the following unified bound of the sum of $\|g_k\|^2$.

Lemma 7 *Under the assumptions, the sum of $\|g_k\|^2$ over $k \in K^0, K^+$ satisfies*

$$F \geq \frac{c\overline{\alpha}}{M}\sum_{k\in K^0}\|g_k\|^2 + C_1\sqrt{\varsigma + \sum_{k\in K^+}\|g_k\|^2} - C_2\ln\left(\varsigma + \sum_{k\in K^+}\|g_k\|^2\right),$$

Proof Since $f(x_{k_{\max}}) \geq f^*$ by Assumption 1 and $K = K^0 \cup K^+$, we have

$$f(x_0) - f^* \geq f(x_0) - f(x_{k_{\max}}) = \sum_{k \in K} \delta_k = \sum_{k \in K^0} \delta_k + \sum_{k \in K^+} \delta_k.$$

From Lemmas 4 and 6, we can further bound as

$$\begin{aligned} f(x_0) - f^* \geq\ & \frac{c\overline{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 - \frac{2}{1-\varepsilon_{\mathrm{f}}} \left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max}\right) \\ & + C_1 \sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - C_1 \sqrt{\varsigma} - C_2 \ln\left(\varsigma + \sum_{k \in K^+} \|g_k\|^2\right) + C_2 \ln(\varsigma). \end{aligned}$$

Substituting eq. (35) yields the desired bound, concluding the proof. $\square$

Now, we establish the following global convergence result for Algorithm 1.

Theorem 1 (Global Convergence) *Under Assumptions 1 to 3, the average of squared gradient norms generated by Algorithm 1 satisfies:*

$$\frac{1}{k_{\max}} \sum_{k=0}^{k_{\max}-1} \|g_k\|^2 = \mathscr{O}\left(\frac{1}{k_{\max}}\right).$$

Proof Let us define the function $\phi : \mathbb{R}_{>0} \to \mathbb{R}$ as

$$\phi(G) := \frac{C_1}{2}\sqrt{G} - C_2 \ln(G). \tag{36}$$

Since $\phi'(G) = \frac{C_1}{4\sqrt{G}} - \frac{C_2}{G}$, its minimum over $G > 0$ is attained at $G^* := \left(\frac{4C_2}{C_1}\right)^2$ with

$$\phi(G^*) = 2C_2\left(1 - \ln\left(\frac{4C_2}{C_1}\right)\right) = 2C_2 \ln\left(\frac{\mathrm{e}C_1}{4C_2}\right),$$

where e is the base of the natural logarithm. Thus, Lemma 7 yields

$$\begin{aligned} F &\geq \frac{c\overline{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} + \phi\left(\sum_{k \in K^+} \|g_k\|^2 + \varsigma\right) \\ &\geq \frac{c\overline{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} + \phi(G^*), \end{aligned}$$

which is equivalent to

$$0 \leq \frac{c\overline{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} \leq F' \tag{37}$$

where

$$\begin{aligned} F' &:= F - \phi(G^*) \\ &= f(x_0) - f^* + \frac{2}{1-\varepsilon_{\mathrm{f}}} \left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max}\right) + C_1\sqrt{\varsigma} - C_2 \ln(\varsigma) - 2C_2 \ln\left(\frac{\mathrm{e}C_1}{4C_2}\right). \end{aligned}$$

As a general fact, we consider the maximization of $x+y$ under the constraint $ax+b\sqrt{y}\le c$ for $a,b>0$, $c\ge 0$ and $x,y\ge 0$. Since $x+y$ is increasing in both variables, we can assume $ax+b\sqrt{y}=c$. On this boundary, we can write $y=\left(\frac{c-ax}{b}\right)^2$, and thus $x+y$ is convex on an interval $0\le x\le c/a$, attaining its maximum at an endpoint. Therefore the maximum of $x+y$ is achieved either at $(x,y)=(c/a,0)$ or at $(x,y)=(0,(c/b)^2)$. Using this fact with eq. (37), we have

$$\frac{1}{k_{\max}}\sum_{k=0}^{k_{\max}-1}\|g_k\|^2=\frac{1}{k_{\max}}\left(\sum_{k\in K^0}\|g_k\|^2+\sum_{k\in K^+}\|g_k\|^2\right)\le\frac{1}{k_{\max}}\max\left(\frac{M}{c\overline{\alpha}}F',\,\frac{4}{C_1^2}F'^2\right),$$

which completes the proof. □

As a remark, the function $\phi(G)$ in eq. (36) is closely related to the Lambert W function [7, 18]. When $x\in[-1/\mathrm{e},0)$, $ye^y=x$ has two real and negative solutions, and the smaller solution is given by the second branch of Lambert W function $y=W_{-1}(x)<0$. We can explicitly solve the equation $\phi(G)=0$ using this function, and then we can obtain a tighter bound in Theorem 1. Since this is not essential for the main discussion, we omit the details here. See [18] for more details.

It is well-known that Theorem 1 implies that the iteration complexity to achieve

$$\min\left\{k\;\middle|\;\|\nabla f(x_k)\|\le\varepsilon_{\mathrm{gtol}}\right\}=\mathscr{O}\left(1/\varepsilon_{\mathrm{gtol}}^2\right).$$

This is a standard convergence guarantee for first-order optimization algorithms applied to L-smooth nonconvex functions for such problems. We interpret these results in light of classical optimization results.

- As for the term from K^0, this is an analogue of the classical gradient descent bound for an error-contaminated function $\overline{f}$. Indeed, for gradient descent with fixed step size $\alpha=1/L$, the sum of squared gradients is bounded as

$$\sum_{j=0}^{k}\|g_j\|^2\le 2L(f(x_0)-f^*)=\frac{2(f(x_0)-f^*)}{\alpha}.$$

- As for the term from K^+, this is analogous to the convergence property of AdaGrad-Norm, which scales as $(f(x_0)-f^*)^2$ [43]. This follows from the fact that F' contains the initial optimality gaps $f(x_0)-f^*$ and $\overline{f}(x_0)-\overline{f}^*$, and its term is quadratic in F'.

4 Practical Choices of Algorithmic Variables

In sections 2 and 3, we introduced Algorithm 1 and established its convergence properties. In this section, we discuss practical choices of algorithmic variables and parameters that lead to fast and stable performance in practice. The quantities to be discussed here are the matrices B_k, the step sizes α_k, and the regularization parameters μ_k.

4.1 Approximate Hessian B_k

We first discuss the construction of the matrices B_k used in Line 9 of Algorithm 1. As mentioned in **??**, we employ an L-BFGS method to form B_k. The purpose of this subsection is to show that B_k can satisfy Assumption 3, the uniform spectral bound, with the modification and selection of curvature pairs.

Construction of B_k: Before stating concrete construction of B_k, we recall the L-BFGS method and the curvature pair notation [4, 25, 31]. For an iteration k, we define

$$s_k := x_{k+1} - x_k, \qquad y_k := g_{k+1} - g_k \tag{38}$$

where g_k and g_{k+1} are the gradients at x_k and x_{k+1}, respectively. Starting from an initial matrix, an approximate Hessian is obtained by successively applying BFGS updates to a sequence of curvature pairs. Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a collection of curvature pairs, where k_i denotes the iteration index at which the i-th pair was generated. Starting from the initial matrix γI with $\gamma = \|y_{k_0}\|^2 / y_{k_0}^\top s_{k_0}$, we define $\mathrm{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1})$ as the matrix obtained by sequentially applying the BFGS update using these pairs in the given order. Here, p denotes the maximum number of stored curvature pairs and is a fixed small integer in the L-BFGS method. When fewer than p curvature pairs are available, the BFGS update is applied using all currently available pairs.

Now, we provide a sufficient condition for Assumption 3 by the following proposition, which is a variant of existing work [12, Theorem 5.5] [14, Lemma 1]. Its proof is deferred to **??**.

Proposition 2 *Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a fixed number of curvature pairs with $s_{k_i} \neq 0$. Assume that there exist constants $0 < \lambda \leq \Lambda$ such that for all $(s, y) \in \{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$,*

$$y^\top s \geq \frac{1}{\Lambda} \|y\|^2, \tag{39}$$

$$y^\top s \geq \lambda \|s\|^2. \tag{40}$$

Define the condition number $\kappa := \Lambda / \lambda$. Then there exist $0 < m < M$ such that

$$mI \preceq \mathrm{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1}) \preceq MI, \tag{41}$$

where

$$M := \gamma + p\Lambda, \qquad m := \left((1 + \sqrt{\kappa})^{2p} \left(\gamma + \frac{1}{\lambda(2\sqrt{\kappa} + \kappa)} \right) \right)^{-1}.$$

Proposition 2 shows that, whenever eqs. (39) and (40) hold for the stored pairs, the resulting BFGS matrix satisfies Assumption 3 as expressed in eq. (41).

Based on Proposition 2, we basically adopt the following rule for forming B_k:

$$B_k = \mathrm{BFGS}(\{(s_{k_i}, \bar{y}_{k_i})\}), \tag{42}$$

where

$$\bar{y}_{k_i} := \theta_i^{\mathrm{damp}} y_{k_i} + (1 - \theta_i^{\mathrm{damp}}) B_{k-1} s_{k_i}$$

with $\theta_i^{\mathrm{damp}} \in [0,1]$. This parameter is determined using damped BFGS update [26, 33] and the indices $\{k_i\}_{i=0}^{p-1}$ are selected so that the modified pairs $(s_{k_i}, \bar{y}_{k_i})$ satisfy eqs. (39) and (40) for some constants λ, Λ. The damped BFGS is a classical technique to ensure $\bar{y}_{k_i}^\top s_{k_i} > 0$, and we adopted its simple limited-memory variant [26].

Since we do not employ Wolfe conditions in the line search, this damping technique plays a central role in ensuring the positive definiteness of B_k. The selection of curvature pairs shares the same spirit as the cautious update [23, 27, 28], which only uses pairs satisfying certain curvature conditions. By Proposition 2, B_k satisfies the uniform spectral bound in Assumption 3. This confirms that the practical construction of B_k is consistent with the theoretical requirements.

Remarks for Practical Implementation: Having established that the proposed construction of B_k satisfies Assumption 3, we now present several remarks on its practical implementation. We begin by describing how to compute the search direction

$$d_k = -(B_k + \mu_k I)^{-1} g_k \tag{43}$$

with B_k in eq. (42). When $\mu_k = 0$, the direction can be computed efficiently by the standard L-BFGS two-loop recursion [25, 31]. For the case $\mu_k > 0$, regularized limited-memory schemes may in principle be employed [22]. Alternatively, one may adopt the simple and robust approximation mentioned in the same reference. Specifically, we approximate B_k in eq. (42) by

$$B_k \approx \mathrm{BFGS}(\{s_{k_i}, \bar{y}_{k_i} + \mu_k s_{k_i}\}_{i=0}^{p-1}) - \mu_k I, \tag{44}$$

so that $B_k + \mu_k I \approx \mathrm{BFGS}(\{s_{k_i}, \bar{y}_{k_i} + \mu_k s_{k_i}\}_{i=0}^{p-1})$. Its inverse can then be computed using the standard two-loop recursion. Although this approximation is inexact, it has been reported to yield nearly identical practical performance while remaining comparatively simple to implement [22]. We adopt this approximation in the this work.

Separately, there exist function-based modified secant conditions [1, 19, 45, 48] that adjust the secant equation using available function values and gradients when those quantities are deemed reliable. Such adjustments aim to improve practical convergence behavior. We partially adopt these function-based modifications following prior work [46, 49]. The suffix `-MS` in section 5 indicates the use of this technique. Because these modifications are well established, we omit low-level implementation details here. Further specifics are available in the cited references and in our open-source implementation**??**.

4.2 Step Size α_k

We next discuss the adjustment of the step size α_k in Line 4 of Algorithm 2.

As indicated in the pseudo-code, we choose α_k using interpolation of the objective function. Moré–Thuente line searches [29] are widely used in quasi-Newton methods [41], and they typically employ cubic or quadratic interpolation to select trial step sizes. In our implementation, we use essentially the same procedure with a slight modification, namely, clipping the trial step size to the interval $[\beta_{\min}\alpha_k, \beta_{\max}\alpha_k]$. For all numerical experiments, we set the Armijo parameter to $c = 10^{-4}$ and choose the interpolation bounds as $\beta_{\min} = 1/16$ and $\beta_{\max} = 15/16$.

Furthermore, when $\mu_k > 0$ and $\alpha_k = 1$, we introduce a one-time adjustment of the trial step size using gradient information. Let d_k be the search direction and g_k and g_{try} the gradients at the current and trial points. If the directional derivative along d_k changes sign, $d_k^\top g_k < 0$ and $d_k^\top g_{\text{try}} > 0$, and the trial gradient is sufficiently aligned with d_k, namely $d_k^\top g_{\text{try}} > 0.5\|d_k\|\|g_{\text{try}}\|$, we rescale α_k once by a secant-like factor,

$$\alpha_k \leftarrow \operatorname{clip}\left(\alpha_k \frac{-d_k^\top g_k}{d_k^\top g_{\text{try}} - d_k^\top g_k}, \beta_{\min}\alpha_k, \beta_{\max}\alpha_k\right),$$

where $\operatorname{clip}(x,a,b) := \min(\max(x,a),b)$ is a clipping function. When objective values are unreliable and backtracking and interpolation do not occur, this correction effectively refines the step size. Since this adjustment is at most once per iteration, the theoretical results in section 3 remain valid.

4.3 Regularization Parameter μ_k

We finally discuss the adjustment of the regularization parameter μ_k in Line 8 of Algorithm 1.

In our implementation, we choose the regularization parameter as

$$\mu_k = \operatorname{clip}\left(\frac{1}{10}\|g_k\|, \frac{1}{100}G_k, G_k\right) \text{ where } G_k := \sqrt{\varsigma + \sum_{j \in K,\, j \le k} \|g_j\|^2},$$

which is consistent with the general form (eq. (7)) with an adaptive factor $\theta_k \in [10^{-2}, 1]$. In practice, we may set $\varsigma = 0$ as long as the initial gradient is nonzero since ς is mainly introduced for avoiding zero division, but to make it consistent with the theory, we set $\varsigma = 10^{-10}$ in all experiments.

When B_k is a good approximation of the Hessian, the regularization parameter μ_k should be small for fast convergence. Since the quantity G_k is strictly increasing with k, it is natural in practice to decrease the effective regularization when sufficient progress in the objective function is achieved. When the difference between the two sides of eq. (6) is larger than 1, we restart the accumulation by resetting K^+ to the empty set and G_k to ς. As long as the number of such restarts is finite, the overall convergence analysis in section 3 remains valid. Since these choices are heuristic, there may be room for further improvement. Especially, further increasing μ_k stabilizes the method when the objective function values are highly noisy, and decreasing μ_k more aggressively may accelerate convergence when the objective function is accurate.

5 Numerical Experiments

5.1 Methods

We compared the following optimization algorithms:

- (`Ours`): Our proposed regularized quasi-Newton method
  - We used Algorithms 1 and 2 with the details in section 4.
  - (`Ours-MS`) means the variant incorporating the modified secant condition described in section 4.1.
- (`Line`): Line-search L-BFGS method
  - Our Python implementation ported from LBFGSpp [32].
  - The step size was determined using the Moré–Thuente line search, closely resembling SciPy's implementation.
  - (`Line-MS`) means the same variant as `Ours-MS`.
- (`Reg`): Regularized L-BFGS method [22]
  - A quadratic regularization-based quasi-Newton method that does not rely on line search and is conceptually closer to trust-region methods.
  - We wrapped the function `regLBFGS` using `solveNonmonotone` from [22].
  - (`Reg-Sec`) means the incorporation of the approximated computation as in eq. (44). This is wrapped function of `regLBFGSsec` from [22].
- (`SciPy`): SciPy's L-BFGS-B method [41]
  - An established L-BFGS method implemented in C and called from Python.
  - We used the default parameters, with the exception that `ftol` was set to 0 to avoid premature termination.
- (`NTQN`): Noise-tolerant quasi-Newton method [37, 44]
  - A method designed to accommodate inexact gradient information, as mentioned in section 1.1.
  - We mainly used the default parameter settings, and slightly modified the stopping criteria to terminate at the desired gradient norm tolerance.

In the following, each method is referred to by the name given in parentheses. All these methods are L-BFGS-based, and we set the number of stored curvature pairs to 10, the default value in SciPy's L-BFGS-B implementation.

All experiments were conducted on a Microsoft Windows 11 Home system (version 10.0, build 26100) equipped with a 13th Gen Intel® Core™ i7-1360P CPU, featuring 12 physical cores and 16 logical processors. All computations were performed using the CPU only.

5.2 Experimental Results

5.2.1 Test Problems and Settings

We evaluated all algorithms on unconstrained problems from the CUTEst benchmark collection [13], implemented in Python using the PyCUTEst interface [11]. All problems were initialized with the default starting points provided by the library. Following the previous work [22], we excluded 17 problems where either the initial point is already stationary or all algorithms are known to fail. This left 220 test problems. To simulate different numerical environments, we tested the following four settings:

- Artificial noise with absolute noise level of 10^{-3} at 64-bit precision (220 problems, $\varepsilon_{\mathrm{f}} = 10^{-2}$)

explanation	min abs value	relative error	ε_{f}
64-bit double precision	2.23×10^{-308}	$2^{-52} \approx 2.22 \times 10^{-16}$	2.22×10^{-9}
32-bit single precision	1.18×10^{-38}	$2^{-23} \approx 1.19 \times 10^{-7}$	1.19×10^{-3}
16-bit half precision	6.10×10^{-5}	$2^{-10} \approx 9.77 \times 10^{-4}$	9.77×10^{-2}

Table 1: The "min abs value" is the smallest positive normalized number, the "relative error" is the maximum relative rounding error, and ε_{f} is the parameter in (**??**) used in our experiments.

- 64-bit floating-point precision (220 problems, $\varepsilon_{\mathrm{f}} = 2.22 \times 10^{-9}$)
- 32-bit floating-point precision (220 problems, $\varepsilon_{\mathrm{f}} = 1.19 \times 10^{-2}$)
- 16-bit floating-point precision (152 problems, $\varepsilon_{\mathrm{f}} = 9.77 \times 10^{-1}$)

Each setting uses a different error rate ε_{f} in the error model (eq. (1)), as summarized in Table 1. The table relates the machine epsilon (maximum relative rounding error) for each floating-point precision to the ε_{f} value used in our experiments. We set ε_{f} equal to or larger than the machine epsilon to account for additional uncertainties in function and gradient evaluations. We also set the maximum number of iterations to $k_{\max} = 15{,}000$ and timed out runs exceeding 10 minutes per problem.

In the reduced precision settings, to simulate lower numerical precision, we first casted the input vector x to the target lower precision, then computed $\overline{f}(x)$ and $\nabla\overline{f}(x)$ using the lower-precision input. Although the internal computations of CUTEst remain in 64-bit precision, the casting of the input vector effectively simulated the impact of reduced precision on function and gradient evaluations. In the 16-bit precision setting, we excluded problems where the initial gradient norm contains `NaN` or `INF` values after casting, resulting in a total of 152 problems.

In the artificial noise setting, we added uniform random noise sampled from $[-10^{-3}, 10^{-3}]$ to both the objective function values and each component of the gradient vectors independently, i.e.,

$$\overline{f}(x) = f(x) + \mathrm{unif}(-10^{-3}, 10^{-3}), \quad \nabla\overline{f}(x) = \nabla f(x) + \mathrm{unif}(-10^{-3}, 10^{-3})\mathbb{1},$$

where $\mathrm{unif}(a,b)$ denotes a uniform random variable in the interval $[a,b]$ and $\mathbb{1}$ is the vector of all ones. This setting is the same as the one in [37].

5.2.2 Performance Profiles

We used performance profiles [9] to compare the algorithms. Let P denote the set of test problems and S the set of solvers. For each problem $p \in P$ and solver $s \in S$, let $t_{p,s} > 0$ denote the number of function and gradient evaluations required to reach a specified gradient norm tolerance $\varepsilon_{\mathrm{gtol}}$. Following common implementations [41], we used the infinity norm for the gradient, though other norms may equally be employed. Note that $t_{p,s}$ is not time in this experiment but the number of evaluations. When a solver fails to converge within the gradient tolerance or exceeds a maximum number of evaluations, we set $t_{p,s} = +\infty$. Then, the performance ratio $r_{p,s}$ is defined as

$$r_{p,s} = \frac{t_{p,s}}{\min_{s' \in S} t_{p,s'}}.$$

For a given threshold $\tau \geq 1$, the performance profile plots the proportion of problems for which $r_{p,s} \leq \tau$, which means that the y-coordinate for solver s at x-coordinate τ is given by

$$\rho_s(\tau) = \frac{1}{|P|} \left| \{ p \in P \mid r_{p,s} \leq \tau \} \right|.$$

A curve that lies above others indicates better overall performance.

5.2.3 Results with Artificial Noise

We first present the results in the artificial noise setting described in section 5.2.1. Figure 1 presents the performance profile. In this regime, the proposed method shows superior robustness compared to other quasi-Newton methods. This result highlights the validity of our theoretical analysis in section 3 and demonstrates the effectiveness of our regularization-based approach in handling noisy objective function values.

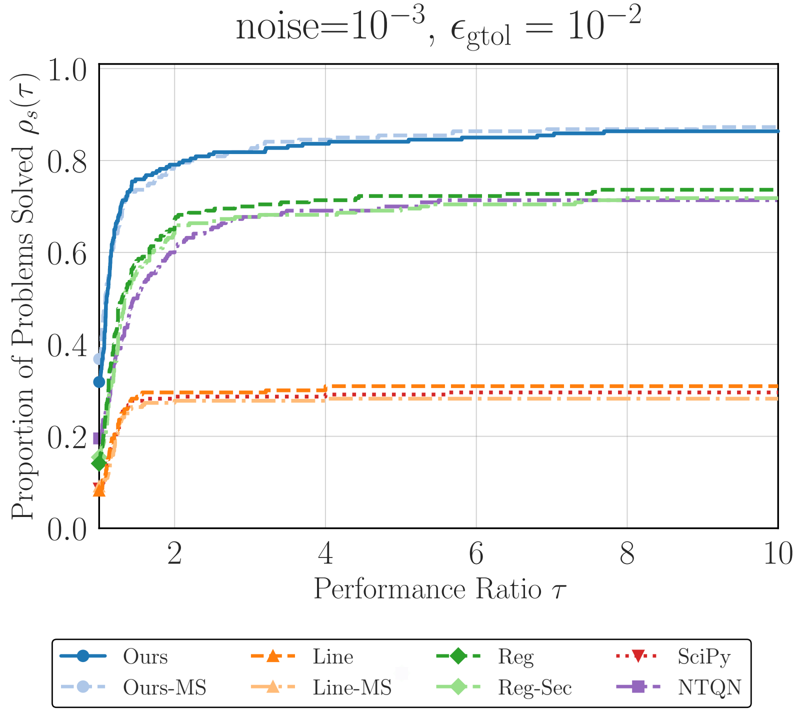
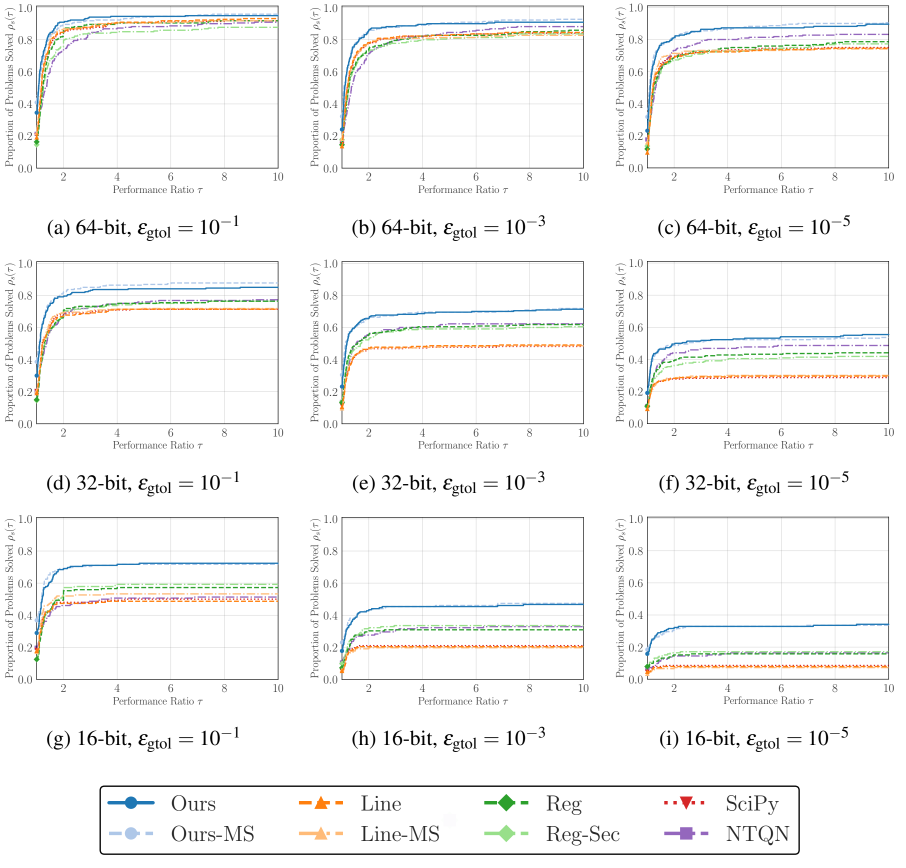


Fig. 1: Performance profile for artificial noise setting (level of 10^{-3}) and $\varepsilon_{\text{gtol}} = 10^{-2}$.



(a) 64-bit, $\varepsilon_{\text{gtol}} = 10^{-1}$ (b) 64-bit, $\varepsilon_{\text{gtol}} = 10^{-3}$ (c) 64-bit, $\varepsilon_{\text{gtol}} = 10^{-5}$

(d) 32-bit, $\varepsilon_{\text{gtol}} = 10^{-1}$ (e) 32-bit, $\varepsilon_{\text{gtol}} = 10^{-3}$ (f) 32-bit, $\varepsilon_{\text{gtol}} = 10^{-5}$

(g) 16-bit, $\varepsilon_{\text{gtol}} = 10^{-1}$ (h) 16-bit, $\varepsilon_{\text{gtol}} = 10^{-3}$ (i) 16-bit, $\varepsilon_{\text{gtol}} = 10^{-5}$

Fig. 2: Performance profiles across different floating-point precisions (64, 32, and 16-bit) and tolerance levels. Each row corresponds to a different precision level, demonstrating the robustness of the proposed method as numerical precision degrades.

5.2.4 Results at Different Precision Levels

We next present performance profiles for each floating-point precision (64-bit, 32-bit, and 16-bit). We evaluated each method at three gradient norm tolerance levels, i.e., $\varepsilon_{\text{gtol}} \in \{10^{-1}, 10^{-3}, 10^{-5}\}$. These tolerance levels represent different optimization regimes, from moderate accuracy to high precision requirements. Figure 2 shows the comprehensive performance profiles across all three precision levels. Our methods (`Ours`) and (`Ours-MS`) demonstrates competitive or superior performance compared to existing methods in the standard 64-bit setting. As numerical precision degrades from 64-bit to 32-bit and further to 16-bit, our regularization-based approach demonstrates improved relative performance. The proposed method maintains stability even when traditional line-search methods encounter difficulties with reduced precision arithmetic.

5.3 Computational Time per Iteration

In this subsection, we present an experiment to compare the per-iteration computational cost of each method. Since the previous experiments focused on the number of function and gradient evaluations, they did not directly reflect the computational overhead of each algorithm. Here, we measured the total execution time required to perform a fixed number of iterations on a simple problem. We conducted experiments on the ill-conditioned quadratic problem

$$\underset{x\in\mathbb{R}^n}{\text{minimize}} \quad \frac{1}{2}\sum_{i=1}^{n} i x_i^2$$

with $n = 10{,}000$, and measured the total execution time. The initial point x_0 was chosen as the all-ones vector. We used memory size $m = 10$ and maximum iterations $k_{\max} = 100$, and we did not set any stopping criteria other than reaching $k_{\max}$. The simplicity of the optimization problem and the small value of $k_{\max}$ are well suited to the purpose of this experiment, as they ensure that the total runtime is dominated by algorithmic overhead rather than function or gradient computation. Timing data were collected after three warm-up runs, followed by 100 recorded trials.

In Figure 3, we report the execution time statistics, and we can observe that our proposed methods (`Ours`) and (`Ours-MS`) exhibit competitive performance compared to other methods. Combining with the results from section 5.2, our methods is competitive or superior in total oracle calls and per-iteration computational cost, indicating practical efficiency. As a side note, although the (`SciPy`) calls optimized C routines, it incurs associated dispatch overhead, producing a larger mean time. The main reason of the (`Reg`) and (`Reg-Sec`) methods taking longer execution time is mainly due to the implementation aspect of the original code.

In Table 2, we report the total number of oracle calls (calls of $\overline{f}$ and $\nabla\overline{f}$) and final objective values $f(x_{k_{\max}})$. The purpose of this table is to confirm that all methods achieved similar convergence behavior within the limited number of iterations. The comparable values in the table indicate that the comparison of execution times is fair and not affected by differences in convergence behavior.

	`Line`	`Ours`	`Line-MS`	`Ours-MS`	`NTQN`	`SciPy`	`Reg-Sec`	`Reg`
# Calls	212	202	212	202	219	212	206	206
$\overline{f}(x_{k_{\max}})$	1.24	1.34	1.24	1.34	1.43	1.24	1.48	1.47

Table 2: The number of oracle calls and final observed objective values for the experiment in section 5.3.

6 Conclusion

In this work, we proposed a regularized quasi-Newton method for noisy nonconvex optimization and established its global convergence properties. We also demonstrated

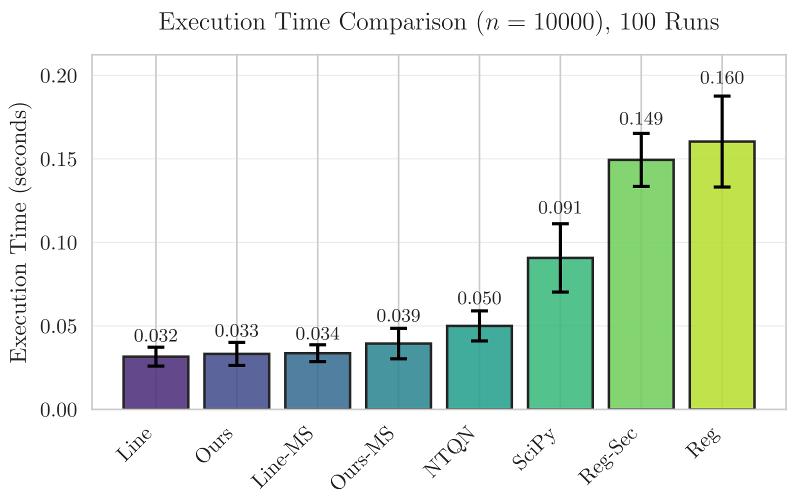


Fig. 3: Mean execution times for the experiment in section 5.3. This plot roughly indicates the extra computational overhead of each method. Error bars represent one standard deviation over 100 runs.

the method's effectiveness through numerical experiments on standard benchmark problems under various noise and precision settings. The results indicate that our method is robust and practically efficient, particularly in noisy or low-precision environments where traditional line-search methods may struggle.

Future work includes the investigation of local convergence properties, the extension to constrained optimization, and the application to practical problems in machine learning and scientific computing. Additionally, exploring adaptive strategies for selecting algorithmic parameters based on problem characteristics could further enhance the method's performance.

Explicitly analyzing the impact of inexact gradient evaluations can also provide valuable insights into the method's robustness and guide further enhancements.

References

1. Babaie-Kafaki S, Ghanbari R, Mahdavi-Amiri N (2010) Two new conjugate gradient methods based on modified secant equations. Journal of Computational and Applied Mathematics 234(5):1374–1386, DOI 10.1016/j.cam.2010.01.052
2. Bellavia S, Gurioli G, Morini B, Toint PL (2021) The Impact of Noise on Evaluation Complexity: The Deterministic Trust-Region Case. DOI 10.48550/arXiv.2104.02519, 2104.02519
3. Berahas AS, Cao L, Choromanski K, Scheinberg K (2021) A Theoretical and Empirical Comparison of Gradient Approximations in Derivative-Free Optimization. DOI 10.48550/arXiv.1905.01332, 1905.01332

4. Berahas AS, Curtis FE, Zhou B (2022) Limited-memory BFGS with displacement aggregation. Mathematical Programming 194(1):121–157, DOI 10.1007/s10107-021-01621-6
5. Carter RG (1993) Numerical Experience with a Class of Algorithms for Nonlinear Optimization Using Inexact Function and Gradient Information. SIAM Journal on Scientific Computing 14(2):368–388, DOI 10.1137/0914023
6. Clancy RJ, Menickelly M, Hückelheim J, Hovland P, Nalluri P, Gjini R (2022) TROPHY: Trust Region Optimization Using a Precision Hierarchy. In: Computational Science – ICCS 2022, Springer, Cham, pp 445–459, DOI 10.1007/978-3-031-08751-6_32
7. Corless RM, Gonnet GH, Hare DEG, Jeffrey DJ, Knuth DE (1996) On the LambertW function. Advances in Computational Mathematics 5(1):329–359, DOI 10.1007/BF02124750
8. Doikov N, Mishchenko K, Nesterov Y (2024) Super-Universal Regularized Newton Method. SIAM Journal on Optimization 34(1):27–56, DOI 10.1137/22M1519444
9. Dolan ED, Moré JJ (2002) Benchmarking optimization software with performance profiles. Mathematical Programming 91(2):201–213, DOI 10.1007/s101070100263
10. Duchi J, Hazan E, Singer Y (2011) Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. Journal of Machine Learning Research 12(61):2121–2159
11. Fowkes J, Roberts L, Bűrmen Á (2022) PyCUTEst: An open source Python package of optimization test problems. Journal of Open Source Software 7(78):4377, DOI 10.21105/joss.04377
12. Gao W, Goldfarb D (2018) Quasi-Newton Methods: Superlinear Convergence Without Line Searches for Self-Concordant Functions. DOI 10.48550/arXiv.1612.06965, 1612.06965
13. Gould NI, Orban D, Toint PL (2015) CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for mathematical optimization. Comput Optim Appl 60(3):545–557, DOI 10.1007/s10589-014-9687-3
14. Gower RM, Goldfarb D, Richtárik P (2016) Stochastic Block BFGS: Squeezing More Curvature out of Data. DOI 10.48550/arXiv.1603.09649, 1603.09649
15. Grapiglia GN, Stella GFD (2022) An adaptive trust-region method without function evaluations. Computational Optimization and Applications 82(1):31–60, DOI 10.1007/s10589-022-00356-0
16. Gratton S, Toint PL (2020) A note on solving nonlinear optimization problems in variable precision. Computational Optimization and Applications 76(3):917–933, DOI 10.1007/s10589-020-00190-2
17. Gratton S, Toint PL (2023) OFFO minimization algorithms for second-order optimality and their complexity. Computational Optimization and Applications 84(2):573–607, DOI 10.1007/s10589-022-00435-2
18. Gratton S, Jerad S, Toint PhL (2024) Complexity of a class of first-order objective-function-free optimization algorithms. Optimization Methods and Software 0(0):1–31, DOI 10.1080/10556788.2023.2296431

19. Hassan BA, Moghrabi IAR (2023) A modified secant equation quasi-Newton method for unconstrained optimization. Journal of Applied Mathematics and Computing 69(1):451–464, DOI 10.1007/s12190-022-01750-x
20. Higham NJ (2002) Accuracy and Stability of Numerical Algorithms. Society for Industrial and Applied Mathematics, DOI 10.1137/1.9780898718027
21. Izycheva A, Darulova E (2017) On sound relative error bounds for floating-point arithmetic. In: Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design, FMCAD Inc, Austin, Texas
22. Kanzow C, Steck D (2023) Regularization of limited memory quasi-Newton methods for large-scale nonconvex minimization. Mathematical Programming Computation 15(3):417–444, DOI 10.1007/s12532-023-00238-4
23. Li DH, Fukushima M (2001) On the Global Convergence of the BFGS Method for Nonconvex Unconstrained Optimization Problems. SIAM Journal on Optimization 11(4):1054–1064, DOI 10.1137/S1052623499354242
24. Li YJ, Li DH (2009) Truncated regularized Newton method for convex minimizations. Computational Optimization and Applications 43(1):119–131, DOI 10.1007/s10589-007-9128-7
25. Liu DC, Nocedal J (1989) On the limited memory BFGS method for large scale optimization. Mathematical Programming 45(1):503–528, DOI 10.1007/BF01589116
26. Lotfi S, de Ruisselet TB, Orban D, Lodi A (2020) Stochastic Damped L-BFGS with Controlled Norm of the Hessian Approximation. DOI 10.13140/RG.2.2.27851.41765/1, 2012.05783
27. Mannel F (2025) A Globalization of L-BFGS and the Barzilai–Borwein Method for Nonconvex Unconstrained Optimization. Journal of Optimization Theory and Applications 204(3):1–34, DOI 10.1007/s10957-024-02565-5
28. Mannel F, Om Aggrawal H, Modersitzki J (2024) A structured L-BFGS method and its application to inverse problems. Inverse Problems 40(4):045022, DOI 10.1088/1361-6420/ad2c31
29. Moré JJ, Thuente DJ (1994) Line search algorithms with guaranteed sufficient decrease. ACM Trans Math Softw 20(3):286–307, DOI 10.1145/192115.192132
30. Nesterov Y, Polyak B (2006) Cubic regularization of Newton method and its global performance. Mathematical Programming 108(1):177–205, DOI 10.1007/s10107-006-0706-8
31. Nocedal J, Wright SJ (1999) Numerical Optimization. Springer, DOI 10.1007/978-0-387-40065-5
32. Okazaki N (2007) libLBFGS: A library of limited-memory broyden-fletcher-goldfarb-shanno (l-BFGS)
33. Powell MJ (1978) Algorithms for nonlinear constraints that use lagrangian functions. Math Program 14(1):224–248, DOI 10.1007/BF01588967
34. Ranganath A, Singhal M, Marcia R (2025) Symmetric Rank-One Quasi-Newton Methods for Deep Learning Using Cubic Regularization. DOI 10.48550/arXiv.2502.12298, 2502.12298
35. Ruan H, Yang W (2024) Adaptive Regularized Quasi-Newton Method Using Inexact First-Order Information. Journal of Computational Mathematics 42(6):1656–1687, DOI 10.4208/jcm.2306-m2022-0279

36. Sheng Z, Yuan G, Cui Z (2018) A new adaptive trust region algorithm for optimization problems. Acta Mathematica Scientia 38(2):479–496, DOI 10.1016/S0252-9602(18)30762-8
37. Shi HJM, Xie Y, Byrd R, Nocedal J (2022) A Noise-Tolerant Quasi-Newton Algorithm for Unconstrained Optimization. SIAM Journal on Optimization 32(1):29–55, DOI 10.1137/20M1373190
38. Sun S, Nocedal J (2023) A trust region method for noisy unconstrained optimization. Mathematical Programming 202(1):445–472, DOI 10.1007/s10107-023-01941-9
39. Ueda K, Yamashita N (2010) Convergence Properties of the Regularized Newton Method for the Unconstrained Nonconvex Optimization. Applied Mathematics and Optimization 62(1):27–46, DOI 10.1007/s00245-009-9094-9
40. Ueda K, Yamashita N (2014) A regularized Newton method without line search for unconstrained optimization. Computational Optimization and Applications 59(1):321–351, DOI 10.1007/s10589-014-9656-x
41. Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, Burovski E, Peterson P, Weckesser W, Bright J, van der Walt SJ, Brett M, Wilson J, Millman KJ, Mayorov N, Nelson ARJ, Jones E, Kern R, Larson E, Carey CJ, Polat I, Feng Y, Moore EW, VanderPlas J, Laxalde D, Perktold J, Cimrman R, Henriksen I, Quintero EA, Harris CR, Archibald AM, Ribeiro AH, Pedregosa F, van Mulbregt P, SciPy 10 Contributors (2020) SciPy 1.0: Fundamental algorithms for scientific computing in python. Nature Methods 17:261–272, DOI 10.1038/s41592-019-0686-2
42. Wang S, Fadili J, Ochs P (2025) Convergence rates of regularized quasi-Newton methods without strong convexity. DOI 10.48550/arXiv.2506.00521, 2506.00521
43. Ward R, Wu X, Bottou L (2020) AdaGrad stepsizes: Sharp convergence over nonconvex landscapes. J Mach Learn Res 21(1):219:9047–219:9076
44. Xie Y, Byrd RH, Nocedal J (2020) Analysis of the BFGS method with errors. SIAM Journal on Optimization 30(1):182–209
45. Yabe H, Ogasawara H, Yoshino M (2007) Local and superlinear convergence of quasi-Newton methods based on modified secant conditions. Journal of Computational and Applied Mathematics 205(1):617–632, DOI 10.1016/j.cam.2006.05.018
46. Yuan G, Wei Z (2010) Convergence analysis of a modified BFGS method on convex minimizations. Computational Optimization and Applications 47(2):237–255, DOI 10.1007/s10589-008-9219-0
47. Zhang H, Ni Q (2018) A new regularized quasi-Newton method for unconstrained optimization. Optimization Letters 12(7):1639–1658, DOI 10.1007/s11590-018-1236-z
48. Zhang J, Xu C (2001) Properties and numerical performance of quasi-Newton methods with modified quasi-Newton equations. Journal of Computational and Applied Mathematics 137(2):269–278, DOI 10.1016/S0377-0427(00)00713-5
49. Zhang JZ, Deng NY, Chen LH (1999) New Quasi-Newton Equation and Related Methods for Unconstrained Optimization. Journal of Optimization Theory and Applications 102(1):147–167, DOI 10.1023/A:1021898630001